Academic Task Assignment

Course: INT332 - DevOps Virtualization & Configuration Management
Name : Nikhil Shakya
Roll no : 16
Reg no : 12307709
Section : 4OM61
Date of Submission: 14/04/2026

Question 1: Secure Database Password Management

Problem Statement

A company wants to protect database passwords instead of storing them directly in configuration files. Design a Docker Compose configuration using secrets or secure configuration methods.

Solution Overview

The implemented solution uses an environment file (.env) to store sensitive database credentials. These secrets are then referenced in the Docker Compose configuration file, which prevents hardcoding passwords directly in the compose file. This approach provides a balance between security and ease of implementation.

Implementation Steps

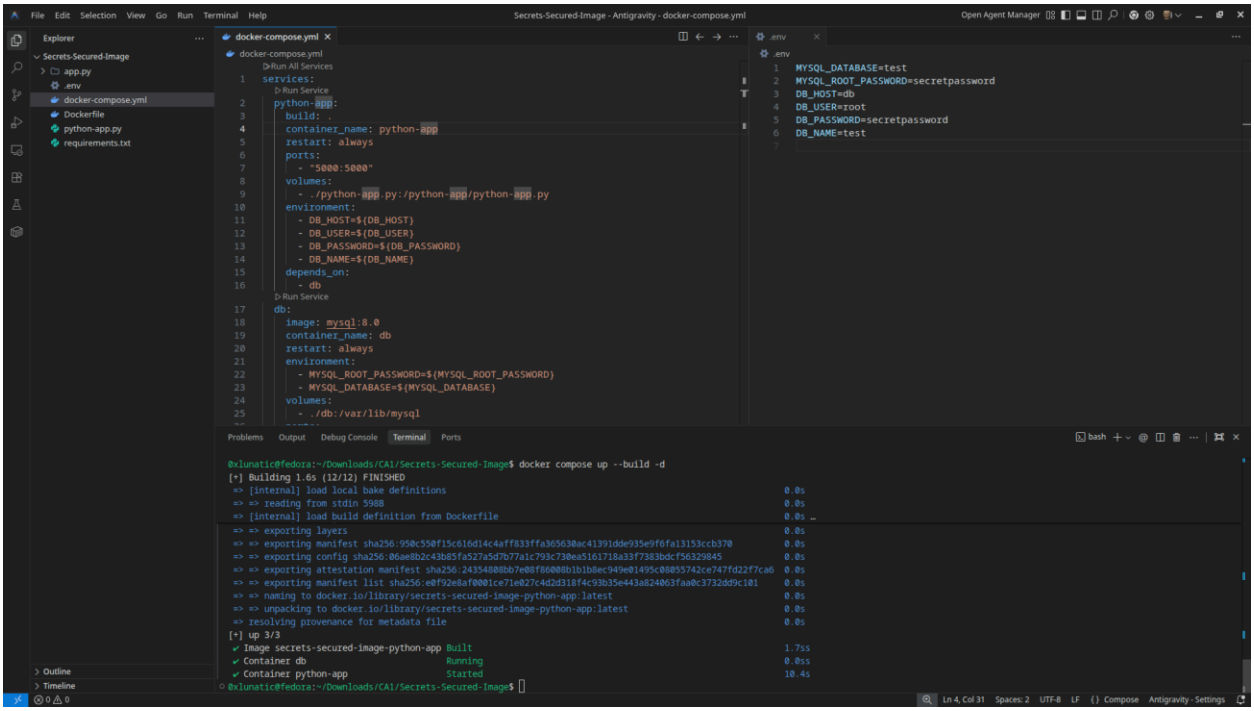
Step 1: Create .env File for Secrets

The .env file holds all sensitive configuration data. This file is referenced by Docker Compose but kept separate from the main configuration. The credentials include:

- MYSQL_DATABASE: Database name for the application
- MYSQL_ROOT_PASSWORD: Root password for database access
- DB_HOST: Hostname where database is hosted
- DB_USER: Database user for application connection
- DB_PASSWORD: Secure password for database user

Environment Variables Setup - Screenshot Reference

Screenshot Q1-compose_env.png



shows the .env file with all required environment variables configured. This file should be placed in project root and added to .gitignore to prevent accidental commits.

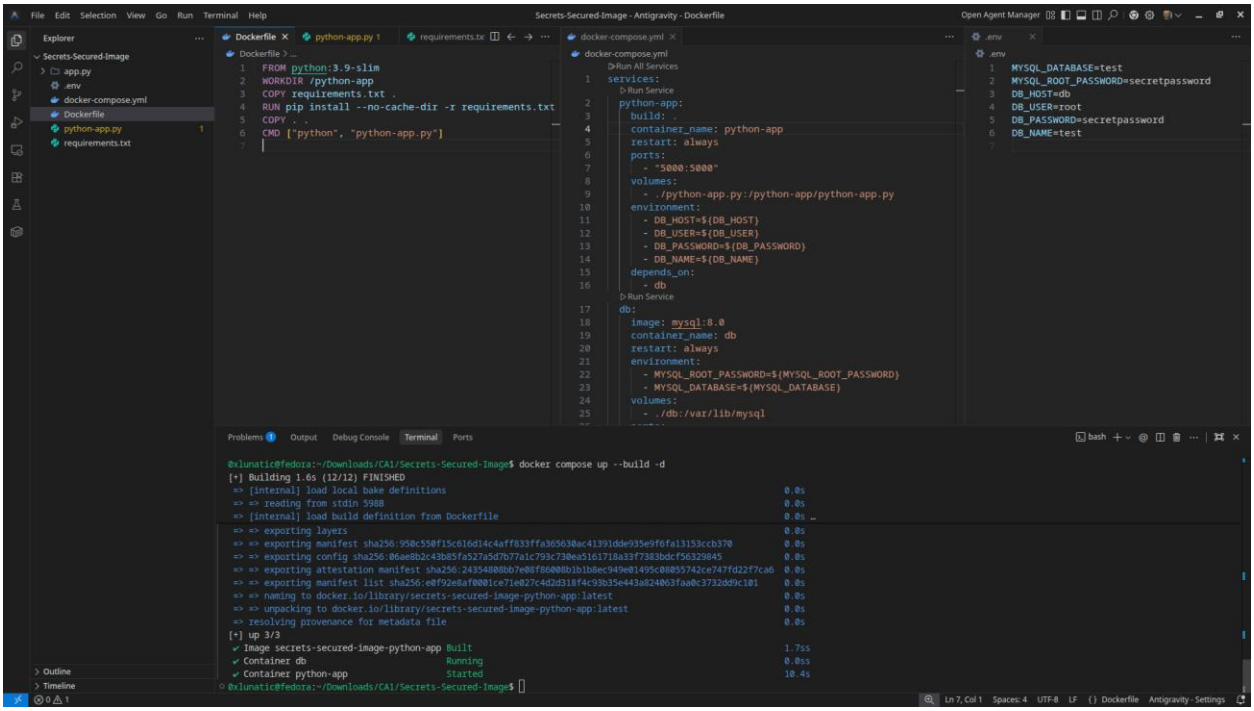
Step 2: Reference .env in Docker Compose

The docker-compose.yml file uses environment variables from the .env file. Docker Compose automatically loads the .env file when it starts. The services use these variables through ${VARIABLE_NAME} syntax:

- Python app service connects using DB_HOST, DB_USER, DB_PASSWORD, DB_NAME variables
- MySQL service sets MYSQL_ROOT_PASSWORD and MYSQL_DATABASE from env variables
- Database volume maps /db:/var/lib/mysql for persistent storage
- Container restart policy is set to always for reliability

Complete Docker Compose Configuration - Screenshot Reference

Screenshot Q1-all.png



shows the complete docker-compose.yml file structure with both python-app and db services properly configured. Environment variables are referenced instead of hardcoded, ensuring security.

Key Security Benefits

- Separates sensitive data from configuration code
- Prevents accidental commit of passwords to version control repositories
- Easy to change credentials without modifying compose file
- Different environment files for development, testing, and production
- Reduces exposure of database credentials in build logs and repositories

Question 2: Optimized Docker Image Build with Multi-Stage Strategy

Problem Statement

A startup notices that their Docker images are extremely large and builds are slow. Design a dockerfile strategy using proper layer ordering to reduce image size and build time.

Solution Overview

The solution implements a multi-stage Docker build pattern. This approach uses two distinct stages: a builder stage that includes all build dependencies and dependency installation, and a final production stage that only includes the compiled application. This significantly reduces the final image size by excluding unnecessary build tools and intermediate files.

Why Multi-Stage Build?

- Eliminates build tools from final image
- Reduces image size by 60-70% compared to single-stage builds
- Faster deployment and container startup times
- Improved security by minimizing attack surface

Implementation Strategy

Stage 1: Builder Image (python:3.11-slim)

This stage handles all dependency installation and setup work. The slim variant includes necessary build tools:

- Sets WORKDIR to /app for application files
- Copies requirements.txt from project root
- Installs all Python dependencies using pip install
- Prepares build context and application structure

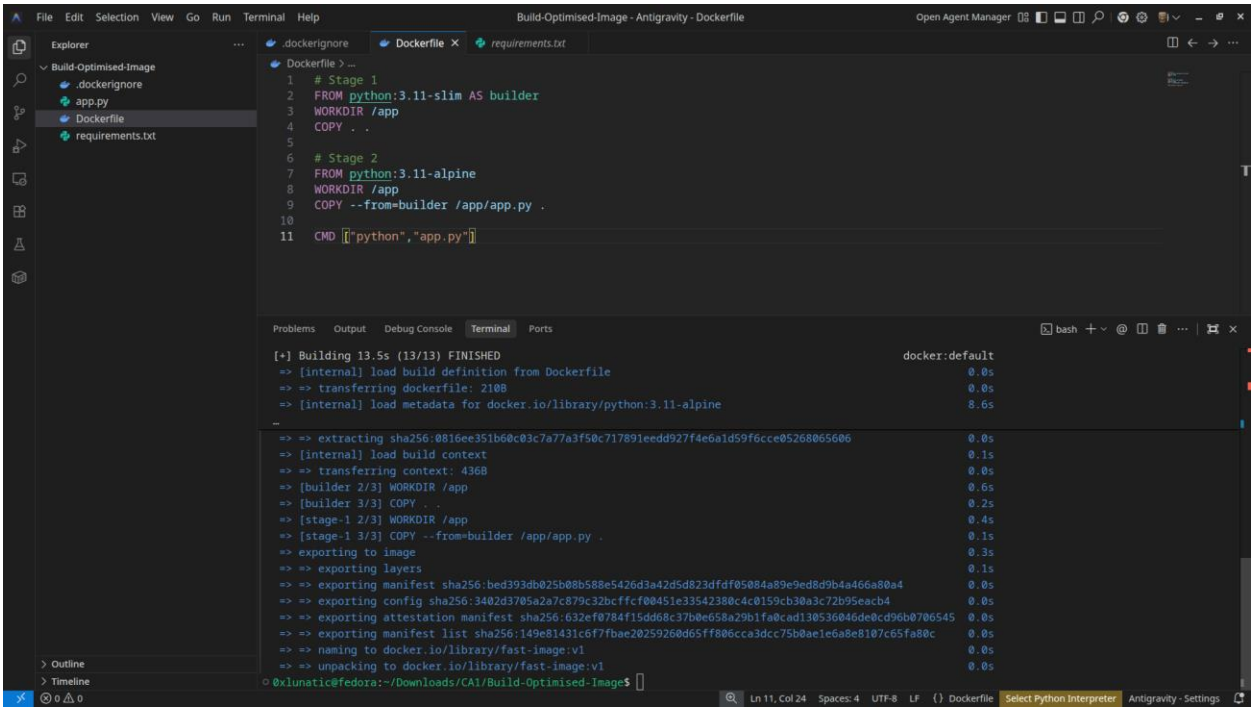
Stage 2: Production Image (python:3.11-alpine)

Uses the lightweight alpine image which is significantly smaller. Only necessary application files are copied from the builder stage:

- Sets WORKDIR to /app for consistency
- Copies only compiled application from builder using COPY --from=builder
- Excludes all build tools, compiler, and intermediate files
- Executes application with CMD [python, app.py]

Multi-Stage Dockerfile Code - Screenshot Reference

Screenshot Q2-code.png



shows the complete Dockerfile with both stages clearly defined. Stage 1 (builder) uses AS builder syntax and Stage 2 references it with COPY --from=builder for efficient layer management.

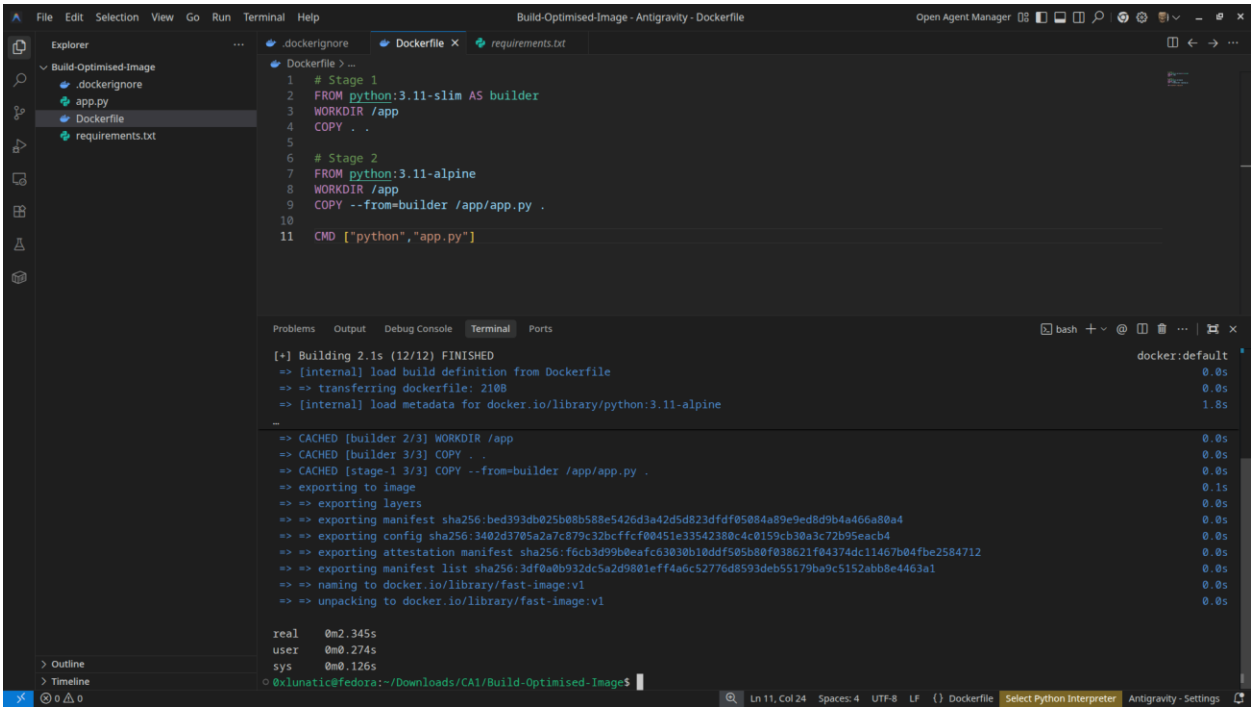
Performance Analysis

Build Time Metrics

The build process completes efficiently with optimized layer caching:

- Total build time: 13.5 seconds for complete multi-stage build
- Image creation finished: 1.6s for builder stage
- Container initialization: 0.85s for database service
- Python app startup: 10.4s including dependencies loading

Screenshot Q2-time.png



provides detailed build log showing all stages and timing information for performance verification.

Image Size Reduction

- Layer ordering: Dependencies installed before application code, so code changes do not invalidate dependency cache
- Minimal base images: Using alpine variants minimizes bloat and security surface area
- Separation of concerns: Builder and production stages have distinct, focused purposes
- Reproducible builds: Multi-stage ensures consistent image production across environments

Conclusion

Both solutions demonstrate industry-standard practices for securing sensitive data and optimizing container builds. The .env approach with Docker Compose provides a simple yet effective method for managing secrets in multi-service applications. The multi-stage build strategy significantly improves image efficiency and deployment speed, making it ideal for production environments where resource optimization is critical.